

# The `BCC_nl_Eshelby_Resist` Mobility Law Binary Nodal Resistance Inside Eshelby Inclusions

KGParadis source-level technical documentation

Code snapshot reviewed 7 August 2026

## Abstract

The mobility selector `BCC_nl_Eshelby_Resist` preserves the original BCC nonlinear constitutive law outside Eshelby inclusions. When the dislocation node itself lies inside any periodic ellipsoidal inclusion, the dimensionless parameter `MobEshelbyResist` multiplies the complete BCC nonlinear resistance, including both force and Jacobian terms. This document derives the binary point test, resistance-scaled force balance, Newton matrix, limiting cases, source integration, and required control-file input. The implementation is a complete standalone source file; neither `MobilityLaw_BCC_nl.cc` nor the existing arm-fraction `BCC_nl_Eshelby` law is changed.

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Selector and implementation files</b>                | <b>2</b> |
| <b>2</b> | <b>Binary node membership</b>                           | <b>2</b> |
| 2.1      | Periodic ellipsoid equation . . . . .                   | 2        |
| 2.2      | Why this is a node test . . . . .                       | 3        |
| <b>3</b> | <b>Outside constitutive law: BCC nonlinear mobility</b> | <b>3</b> |
| 3.1      | Thermally activated screw relation . . . . .            | 3        |
| 3.2      | Nodal BCC nonlinear resistance . . . . .                | 3        |
| <b>4</b> | <b>Inside resistance multiplier</b>                     | <b>4</b> |
| <b>5</b> | <b>Interpretation and limits</b>                        | <b>4</b> |
| <b>6</b> | <b>Control-file input</b>                               | <b>5</b> |
| <b>7</b> | <b>Registration and restart behavior</b>                | <b>6</b> |
| <b>8</b> | <b>Verification and cautions</b>                        | <b>6</b> |

# 1 Selector and implementation files

The control-file selector is

```
mobilityLaw = "BCC_nl_Eshelby_Resist".
```

Its independent entry points are

```
InitMob_BCC_nl_Eshelby_Resist(Home_t *),
Mobility_BCC_nl_Eshelby_Resist(Home_t *,Node_t *,MobArgs_t *).
```

| File                                     | Responsibility                                                                                        |
|------------------------------------------|-------------------------------------------------------------------------------------------------------|
| src/MobilityLaw_BCC_nl_Eshelby_Resist.cc | Complete BCC nonlinear solver, binary ellipsoid membership test, and resistance scaling.              |
| include/Mobility.h                       | Guarded enum, prototypes, and selector registration.                                                  |
| src/DisableUnneededParams.cc             | Preserves BCC nonlinear parameters, pressure, inclusion file, and MobEshelbyResist in restart output. |
| makefile.srccs and src/makefile.dep      | Object registration and dependency closure.                                                           |

The implementation is enclosed by **ESHELBY**. Without that build flag the translation unit is empty and the selector is not registered.

## 2 Binary node membership

### 2.1 Periodic ellipsoid equation

Let the node position be  $\mathbf{x}$ . Inclusion  $p$  has center  $\mathbf{C}_p$ , positive semi-axes  $a_{p0}, a_{p1}, a_{p2}$ , and principal directions  $\mathbf{e}_{p0}, \mathbf{e}_{p1}$ . The third direction is

$$\mathbf{e}_{p2} = \frac{\mathbf{e}_{p0} \times \mathbf{e}_{p1}}{\|\mathbf{e}_{p0} \times \mathbf{e}_{p1}\|}. \quad (1)$$

The source first applies the ParaDiS minimum-image operation,

$$\mathbf{d}_p = \text{ZImage}(\mathbf{x} - \mathbf{C}_p), \quad (2)$$

then evaluates the squared unit-sphere coordinate

$$\chi_p(\mathbf{x}) = \sum_{k=0}^2 \left( \frac{\mathbf{e}_{pk} \cdot \mathbf{d}_p}{a_{pk}} \right)^2. \quad (3)$$

This is the same ellipsoid-to-sphere transform used by `src/SegPartIntersect.cc`.

The binary indicator is

$$I(\mathbf{x}) = \begin{cases} 1, & \min_p \chi_p(\mathbf{x}) \leq 1 + 10^{-12}, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

The tolerance classifies a node on the particle boundary as inside. Locally owned and ghost inclusions are both scanned through

```
home->totInclusionCount.
```

Overlapping particles do not compound the resistance: membership in any one particle sets  $I = 1$ .

## 2.2 Why this is a node test

This law deliberately does not use segment–particle intersection lengths. A node inside a particle is scaled even if its connected arms immediately leave the particle. Conversely, a node outside remains on ordinary BCC nonlinear mobility even when one of its arms crosses a particle.

This differs from `BCC_n1_Eshelby`, which computes an endpoint-weighted covered fraction for every arm and uses a `BCC_0` tensor on covered portions. The two selectors implement different physical models and coexist.

## 3 Outside constitutive law: BCC nonlinear mobility

### 3.1 Thermally activated screw relation

The standalone file contains its own copy of the original BCC nonlinear helpers and initializer. Define

$$p = \text{MobExpP}, \quad q = \text{MobExpQ}, \quad \alpha_0 = \text{MobAlpha}, \quad H = \frac{\text{MobDeltaH0}}{k_B T},$$

and  $P_{\text{GPa}} = P/10^9$ . Initialization computes

$$\mathcal{L} = AC_0 \left( 1 - C \frac{T}{T_m} + DP_{\text{GPa}} \right), \quad (5)$$

$$y = \alpha_0^p,$$

$$\mathcal{N} = \frac{\mathcal{L}}{\exp[-H(1-y)^q] [1 + pqH(1-y)^{q-1}y]}, \quad (6)$$

$$V_0 = \alpha_0 \mathcal{L} - \alpha_0 \mathcal{N} \exp[-H(1-y)^q]. \quad (7)$$

Here  $A, C_0, C, D, T_m$  correspond to `MobCoeffA`, `MobCoeffC0`, `MobCoeffC`, `MobCoeffD`, and `meltTemp`.

For a screw Burgers family, let

$$M_L = \frac{\mathcal{L}}{b_{\text{mag}}}, \quad M_N = \frac{\mathcal{N}}{b_{\text{mag}}}, \quad v_0 = \frac{V_0}{b_{\text{mag}}}, \quad (8)$$

$$f_P = \text{MobPeierls} \|\mathbf{b}\|, \quad \omega = \frac{M_N}{f_P}, \quad v_{\text{tr}} = \alpha_0 M_L - v_0. \quad (9)$$

Below the transition, the scalar screw glide force is the numerical inverse of

$$\boxed{v = \omega f \exp[-H(1 - (f/f_P)^p)^q], \quad 0 \leq v < v_{\text{tr}}.} \quad (10)$$

Above the transition the continuation is

$$f(v) = \frac{(v + v_0)f_P}{M_L}. \quad (11)$$

### 3.2 Nodal BCC nonlinear resistance

For arm  $i$ , let  $L_i$  be its length,  $\mathbf{t}_i$  its unit tangent, and  $\mathbf{b}_i$  its Burgers vector. Its projected lengths are

$$L_{s,i} = \frac{|\mathbf{r}_i \cdot \mathbf{b}_i|}{\|\mathbf{b}_i\|}, \quad L_{e,i} = \sqrt{\max(0, L_i^2 - L_{s,i}^2)}. \quad (12)$$

The source groups collinear screw Burgers vectors. Its complete unscaled nodal resistance may be written

$$\boxed{\mathcal{R}_{\text{nl}}(\mathbf{v}) = D_{\text{lin}}\mathbf{v} + \sum_g C_g \mathbf{F}_{s,g}(\mathbf{v})}, \quad (13)$$

where

$$C_g = \frac{1}{2} \sum_{i \in g} L_{s,i}. \quad (14)$$

$D_{\text{lin}}$  contains the  $L_e/2$  edge tensors and  $L/2$  junction tensors. The nonlinear screw force  $\mathbf{F}_{s,g}$  contains the scalar law in Equations (10)–(11) in the plane normal to  $\mathbf{b}_g$ , plus line drag along  $\mathbf{b}_g$ .

## 4 Inside resistance multiplier

Let

$$\alpha = \text{MobEshelbyResist} > 0 \quad (15)$$

and define the nodal scale

$$\boxed{\beta(\mathbf{x}) = 1 + (\alpha - 1)I(\mathbf{x})}. \quad (16)$$

Thus  $\beta = 1$  outside and  $\beta = \alpha$  inside. The new nodal force balance is

$$\boxed{\mathbf{r}(\mathbf{v}) = \mathbf{F}_{\text{node}} - \beta \mathcal{R}_{\text{nl}}(\mathbf{v}) = \mathbf{0}}. \quad (17)$$

Its Newton Jacobian is

$$\boxed{J(\mathbf{v}) = -\beta \left[ D_{\text{lin}} + \sum_g C_g \frac{\partial \mathbf{F}_{s,g}}{\partial \mathbf{v}} \right]}. \quad (18)$$

The source assembles ordinary BCC\_nl first. Immediately before the one-screw shortcut and Newton loop, it multiplies every component of `df_lindv` and every populated `dfndfscrew[g]` by  $\beta$ . This scales linear edge/junction drag, nonlinear screw force, derivative, line solve, and returned inverse Jacobian exactly once.

Equation (17) also gives the useful identity

$$\boxed{\mathbf{r}_\alpha(\mathbf{v}; \mathbf{F}) = \alpha \mathbf{r}_1(\mathbf{v}; \mathbf{F}/\alpha) \quad \text{for an inside node.}} \quad (19)$$

Therefore the inside solution is ordinary BCC\_nl evaluated at an effective force  $\mathbf{F}/\alpha$ ; no new screw interpolation is introduced.

## 5 Interpretation and limits

| Condition                          | Result                                       |
|------------------------------------|----------------------------------------------|
| $I = 0$                            | Exact original BCC_nl execution and algebra. |
| $I = 1, \alpha = 1$                | Exact original BCC_nl despite being inside.  |
| $I = 1, \alpha > 1$                | Increased resistance and reduced mobility.   |
| $I = 1, 0 < \alpha < 1$            | Reduced resistance and increased mobility.   |
| $\alpha \leq 0$ , NaN, or infinity | Rejected during mobility initialization.     |
| Empty <code>inclusionFile</code>   | $I = 0$ for every node; ordinary BCC_nl.     |

`MobEshelbyResist` is dimensionless. It is not a velocity and not an absolute mobility. It follows the established ParaDiS Eshelby convention

$$B_{\text{inside}} = \alpha B_{\text{outside}}.$$

When `useLabFrame` is enabled, Equation (4) is still evaluated in simulation coordinates before any crystal-frame mobility rotation. The force, velocity, arm, and Burgers vectors then follow the ordinary BCC\_nl rotation path.

## 6 Control-file input

The following block can replace the material/mobility section of a BCC nonlinear control file. The numerical values are the existing tantalum-like defaults and must be replaced by the calibration for the intended material.

```
# Enable particle geometry and select the binary nodal law
mobilityLaw = "BCC_nl_Eshelby_Resist"
inclusionFile = "tests/screwEsh.dat"
materialTypeName = "BCC"

# Dimensionless inside drag multiplier
# 1.0: no change; values greater than 1 reduce inside mobility
MobEshelbyResist = 1.0e5

# Thermodynamic state
TempK = 600.0
pressure = 0.0

# BCC_nl linear mobilities outside and inside
# Inside values are converted to drag and multiplied by MobEshelbyResist
MobGlide = 1.0e2
MobLine = 1.0e4
MobClimb = 1.0e-5

# BCC_nl nonlinear screw parameters
MobPeierls = 3.240000e8
MobDeltaH0 = 1.728748e-19
MobAlpha = 9.661236e-1
MobExpP = 0.5
MobExpQ = 1.23
MobCoeffA = 1.525
MobCoeffC = 2.032370e-1
MobCoeffC0 = 2.048000e3
MobCoeffD = 1.626950e-4
meltTemp = 3.170683e3

# Optional nonlinear-property files; empty uses the values above
MobFileNL = ""
```

```
ecFile = ""
meltTempFile = ""
```

For the repository’s screw/inclusion data, a one-step serial smoke test is

```
bin/paradis -maxstep 1 -s -d tests/screwEsh.data \
  path/to/BCC_nl_Eshelby_Resist.ctrl
```

Unlike the BCC\_0-inside hybrid, this law does not read `MobScrew` or `MobEdge`. Its only new constitutive input is `MobEshelbyResist`; all other mobility inputs are inherited from `BCC_nl`.

## 7 Registration and restart behavior

The selector is registered under `ESHELBY` with `BCC`, non-planar, no-fuzzy-plane, and no-cross-slip attributes matching `BCC_nl`. `src/DisableUnneededParams.cc` retains:

- `inclusionFile` and `MobEshelbyResist`;
- `pressure`, `TempK`, and all `BCC` nonlinear constants;
- `MobGlide`, `MobLine`, and `MobClimb`;
- optional `MobFileNL`, `ecFile`, and `meltTempFile`.

No new parameter binding is needed. `MobEshelbyResist` and `inclusionFile` are already defined by the existing parameter layer.

## 8 Verification and cautions

The implementation was checked with:

- syntax compilation with and without `ESHELBY`;
- complete serial links in both build modes;
- symbol inspection showing the selector and public functions only in the `ESHELBY` binary;
- a one-step ParaDiS run using `tests/screwEsh.data` and `tests/screwEsh.dat`;
- source-level checks of  $\beta = 1$ , outside, and Equation (19);
- periodic, rotated-ellipsoid, boundary, and overlap analysis of the point-membership equation.

The membership loop scans the locally available inclusion array for every mobile node. This guarantees that node-only and zero-length-arm cases do not depend on segment intersection lists, but its cost grows with the number of local and ghost inclusions. If very large particle populations make this dominant, a spatial point-query index can replace the scan without changing Equations (4)–(18).

The copied `BCC_nl` solver intentionally preserves the original law’s numerical branches so the outside limit remains exact. Baseline nonlinear-solver changes should be applied separately and consistently to both source files.